

실시간 채팅 서비스

HTTP/WebSocket 혼합 환경의 상태 정합성 해결

WHY

프로젝트 선택 배경

REST API(Stateless)와 WebSocket(Stateful) 공존 시 발생하는 데이터 정합성 문제를 직접 설계하고 해결하기 위해 시작했다.

단순 "채팅 구현"이 아닌 구조적 문제 해결이 목표

WHAT

해결하려는 핵심 문제

문제 1: Race Condition

동시 입장 요청 시 정원 초과 발생

문제 2: 세션 누수

비정상 종료 시 자원 미회수

Success

목표 설정

목표 1: 데이터 정합성

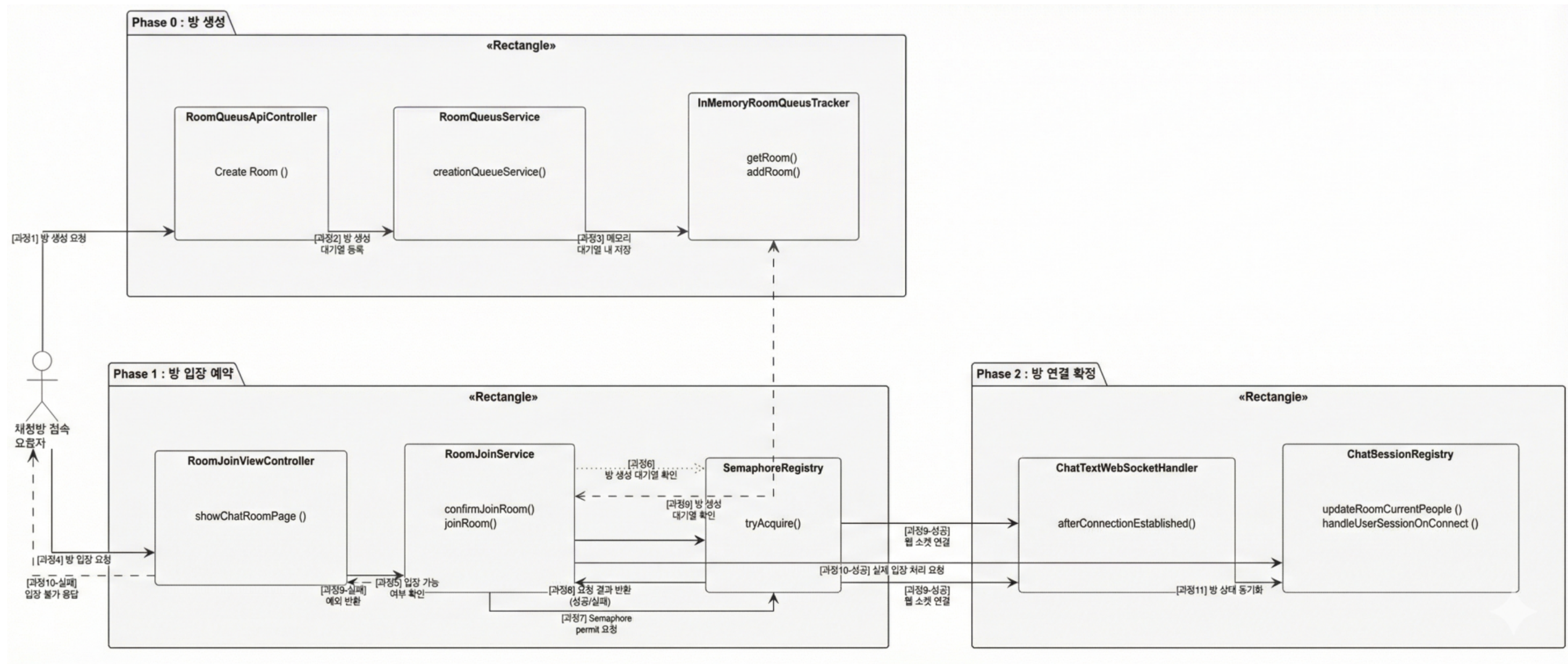
동시 접속에서 정원 초과 0건

목표 2: 자원 효율성

유령 세션 자동 정리

시스템 아키텍처

2-Phase Entry: 입장 예약(Phase 1)과 연결 확정(Phase 2)의 분리를 통한 세션 자원 누수 방지



설계 결정 1

2-Phase 분리 구조

문제 :

입장 허가 후 WebSocket 연결 실패 시 Permit이 회수되지 않는 자원 누수 발생

해결 :

- Phase 1에서 Permit 획득 시 TTL 타임스탬프 기록
- Phase 2 연결 실패 시 스케줄러(30초 주기)가
- TTL 초과 자원을 자동 회수

설계 결정 2

Semaphore 동시성 제어

문제 :

동시 입장 요청 시 check-then-act 패턴의 비원자성으로 인한 Race Condition 발생

해결 :

- Semaphore.tryAcquire() 원자적 Permit 획득
- 정원 초과 오류 0건 (2,000건 동시 요청 테스트)

Synchronized 대비 15% 빠른 413.1μs

설계 결정 1

SessionKey 상태 동기화

문제 :

새로고침/중복탭 시 이벤트 비순차 도착으로 클라이언트-서버 간 상태 불일치 발생

해결 :

- sessionKey 존재 여부 + 서버 저장값 일치 검증
- 최초입장 / 새로고침 / 중복탭을
- 명확히 구분하여 각각 다른 로직으로 처리

트러블슈팅 1: Race Condition 해결

HTTP/WebSocket 혼합 환경의 동시성 문제 해결

문제 인식

check-then-act 으로써 동일한 방 내 다수의 사용자가 동시 접속할 때에 발생하는 RaceCondtion 문제 실측

경합 발생
1,839건
91.95%

값 불일치 : 127 건
상태 전이: 183건
경합 91.95% → 4가지 정합성 오류 연쇄 발생

정원 초과: 1건

경합이 근본 원인이며 정원 초과는 최종 결과

분석 : check-then-act 패턴의 비원자성

Thread A :

정상

Thread B :

경합 발생

Thread C :

3중 경합 발생

임계 구역 내 동시성이 보장 되어 있지 않음으로 인원확인 후 입장허용 간격 내 다른 스레드 개입으로 인한 정원초과 발생

원인 코드 : IF-ELSE

```
문제: check-then-act 비원자성

// RoomJoinServiceIfElse.java - 경합 발생 구간
if (chatRoom.get().getCurrentPeople() >= chatRoom.get().getMaxPeople()) {
    throw new RoomBadJoinFullException("방 정원이 가득 찼습니다.");
} else {
    // ❌ check와 act 사이 다른 스레드 개입 가능
    chatSessionRegistry.incrementCurrentPeople(roomNumber);
}
```

check-then-act 결함

- 조건 확인 ≠ 상태 변경
- 두 연산 사이 스레드 개입 가능
- 정원 5명 방에 6명 입장 발생

if-else로 동시성 보장 불가

해결 : Semaphore.tryAcquire()

```
해결: Semaphore.tryAcquire()

// RoomJoinServiceSemaphore.java - 원자적 연산
boolean acquired = semaphoreRegistry.tryAcquire(roomNumber, userId);
if (!acquired) {
    throw new RoomBadJoinFullException("방 정원이 가득 찼습니다.");
}
// ✅ 확인+획득이 단일 원자적 연산으로 완료
```

CAS 메커니즘 선택 근거

- 확인+획득 원자적 연산 통합
- Permit 개수 = 방 정원
- 경합시에도 정원초과 차단

tryAcquire()로 비원자성 해결

검증 : 2,000건 동시 요청 테스트

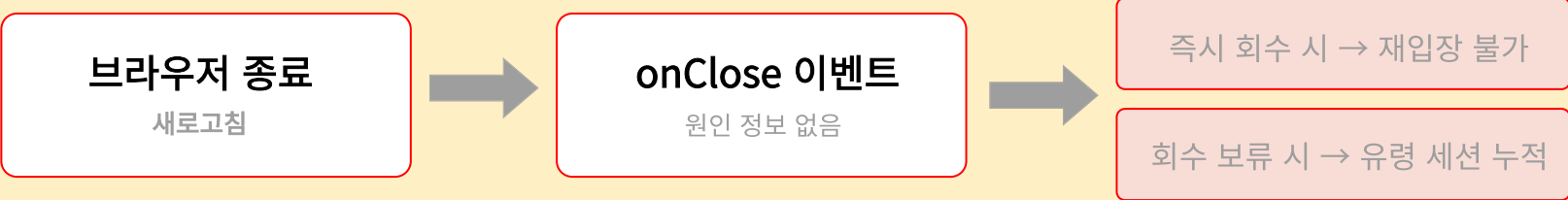
규칙	IF-ELSE (check-then-act)	Synchronized	ReentrantLock	Semaphore
경합	1839	0	0	974 - 정상 동작
값불일치	127	0	0	제외 - 정상 동작
상태전이	183	0	0	제외 - 정상 동작
정원초과	1	0	0	0

Semaphore의 경합 974건은 Permit 범위 내 의도된 동시 접근(동시성 활용률 48.7%). 값불일치·상태전이는 CAS 기반 비순차적 처리 특성으로 분석 대상에서 제외. 핵심 검증 지표인 정원초과 0건으로 비즈니스 규칙 100% 준수 확인.

트러블슈팅 2: 세션 누수 방지

HTTP/WebSocket 혼합 환경의 자원 생명주기 관리

문제 정의



사용자가 명시적 나가기 요청 없이 브라우저를 닫거나 새로고침할 때, 서버는 해당 사용자가 완전히 떠난 것인지 잠시 후 재접속할 것인지 즉시 판단할 수 없다.

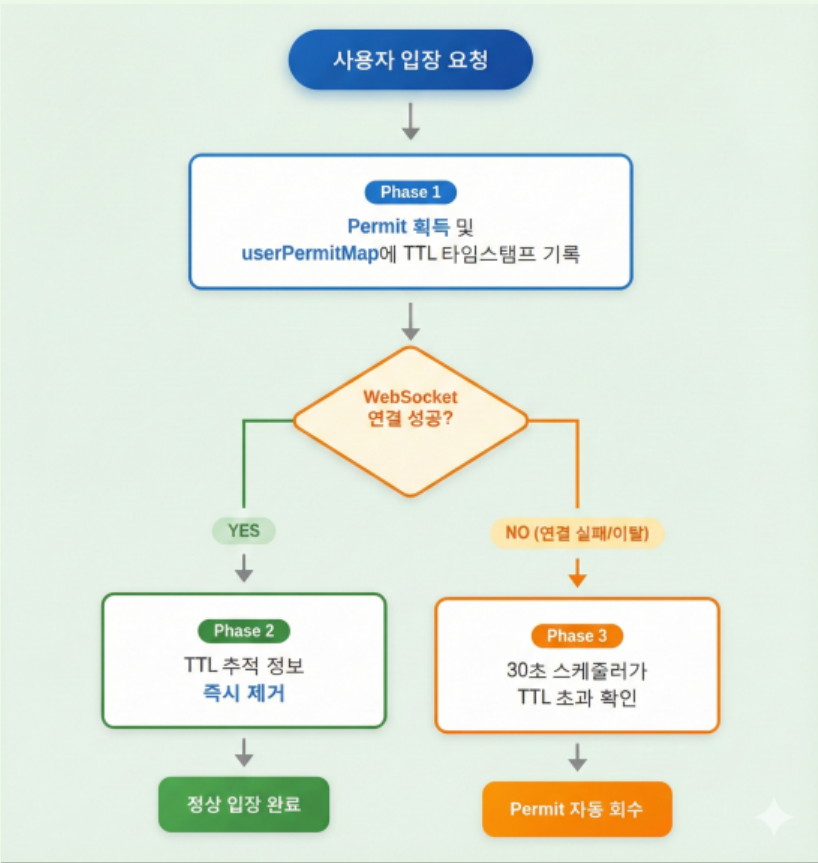
문제 정의

문제의 본질은 '임시 상태'와 '영구 이탈'을 구분할 기준의 부재에 있다.

- WebSocket onClose 이벤트는 종료 원인을 구분하지 못함
- 클라이언트 JS 기반 전달 방식은 사용자 조작에 취약
- 서버 측에서 시간 기반으로 해결해야 함

"즉시 판단 불가능한 상태는 일정 시간 유예 후 판단한다"

해결 : TTL 기반 3단계 메커니즘



[표] TTL 기반 3단계 메커니즘

단계	시점	동작	컴포넌트
Phase 1	Permit 획득 시	TTL 타임스탬프 기록	SemaphoreRegistry
Phase 2	WebSocket 연결 성공	TTL 추적 정보 제거	WebSocketHandler
Phase 3	30초 주기 스케줄러	TTL 초과 Permit 회수	ChatServiceScheduler

SemaphoreRegistry.java - tryAcquire()

```
SemaphoreRegistry.java - tryAcquire()
// Permit 획득 성공 시 TTL 타임스탬프 기록
public boolean tryAcquire(int roomId, String userId) {
    Semaphore sem = semaphoreMap.get(roomId);
    if (sem != null && sem.tryAcquire()) {
        userPermitMap.put(
            new UserRoomKey(userId, roomId),
            System.currentTimeMillis());
        return true;
    }
    return false;
}
```

ChatServiceScheduler.java - clearExpiredUserPermits()

```
ChatServiceScheduler.java - clearExpiredUserPermits()
// 30초마다 TTL 초과 사용자의 Permit 회수
@Scheduled(fixedRate = 30000)
public void clearExpiredUserPermits() {
    long now = System.currentTimeMillis();
    for (var entry : semaphoreRegistry.getUserPermitMap().entrySet()) {
        if (now - entry.getValue() > TTL_LIMIT_MS) {
            semaphoreRegistry.releasePermitOnly(entry.getKey().getRoomId());
            semaphoreRegistry.removePermitTracking(entry.getKey());
        }
    }
}
```

검증 : 새로고침 시 발생하는 세션 누수 해결

문제 : 새로고침 시 established 이벤트가 이전 세션의 close 이벤트보다 먼저 도착

해결 : 최대 100ms간 5ms 간격 폴링으로 close 처리 완료 대기 후 새 세션 등록

즉시 회수 시 -> 재입장 불가

```
// close 이벤트 처리 완료까지 최대 100ms 대기
int maxWaitMs = 100, waited = 0;
while (roomUserVOMap.containsKey(roomNum) &&
    roomUserVOMap.get(roomNum).containsKey(userId) &&
    waited < maxWaitMs) {
    Thread.sleep(5); // 5ms 간격 폴링
    waited += 5;
}
// 이전 세션 정리 완료 후 새 세션 등록 진행
```

성과 검증: Semaphore 선택의 정당성

4종 동시성 메커니즘 비교 테스트 (2,000건 동시 요청)

[테스트 결과]

메커니즘	정원 초과	평균 시간	평가
IF-ELSE	1건	189.0μs	사용 불가
Synchronized	0건	486.4μs	기준선
ReentrantLock	0건	2,464.7μs	407% 느림
Semaphore	0건	413.1μs	15% 빠름

결론 : Semaphore가 Synchronized와 동일한 안전성(정원 초과 0건)을 보장하면서 15% 빠른 성능을 달성하여, 안전성과 성능의 최적 균형점으로 선택했다.

세마포어 선택 근거

기존 방식의 한계

- Synchronized: 상호 배제로 순차 처리 강제 → 성능 병목
- IF-ELSE: 빠르지만 정원 초과 발생 → 비즈니스 규칙 위반

Semaphore의 해법

- Permit 기반 자원 분배로 동시성 제어 + 높은 처리량
- tryAcquire() 즉시 반환으로 무의미한 대기 제거
- CAS 기반 Lock-Free 알고리즘

Semaphore 선택 기술적 근거

Semaphore는 Look-free 알고리즘으로 임계구역에 대한 점유를 하지 않음으로써 성능 저하를 최소한으로 구현

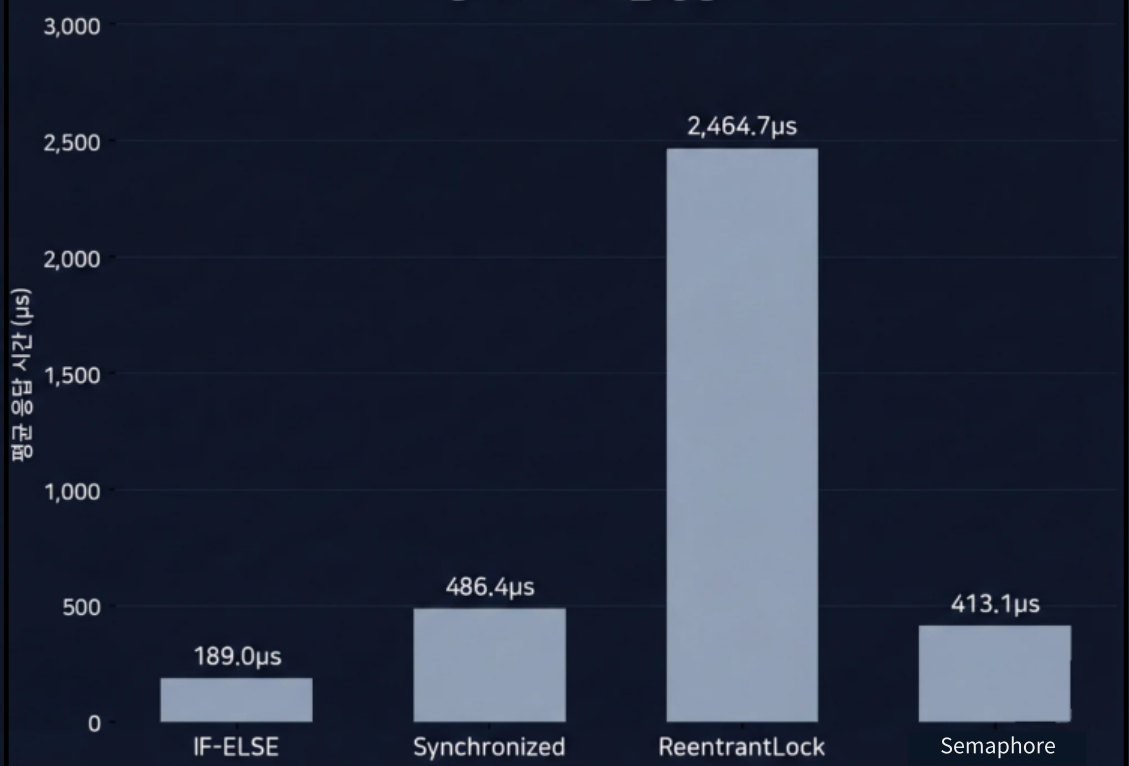
1단계 : 비즈니스 규칙 (100% 보장)

정원 초과 0건 · Permit 기반 원자적 제어

2단계: 성능 최적화 (48.7%)

CAS 기반 동시 접근 허용 · Lock-Free 알고리즘

동기화 메커니즘 성능 비교



규칙 3: 정원 초과 비교

